



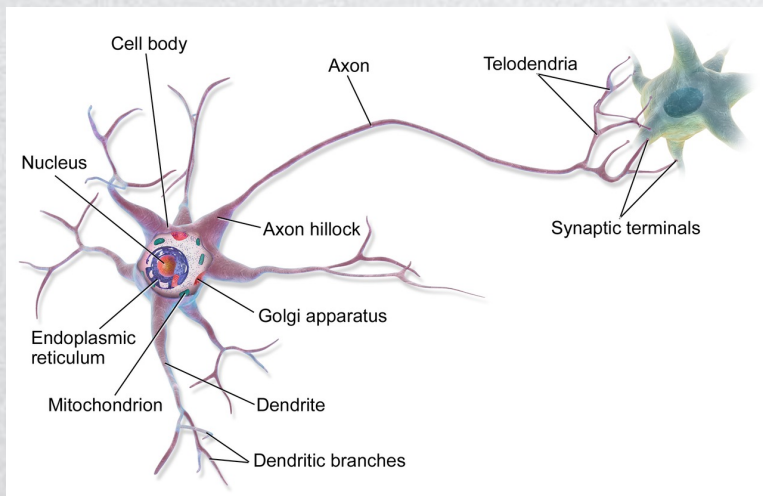
《人工智能与Python程序设计》——图像与卷积



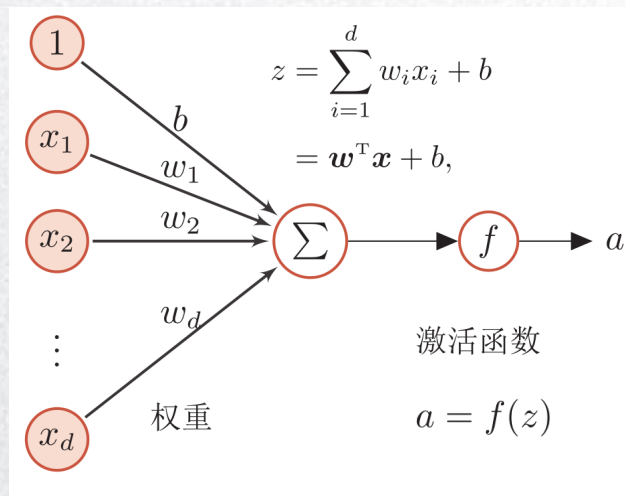
人工智能与Python程序设计 教研组

神经网络概述

• 生物神经元 v.s. 人工神经元



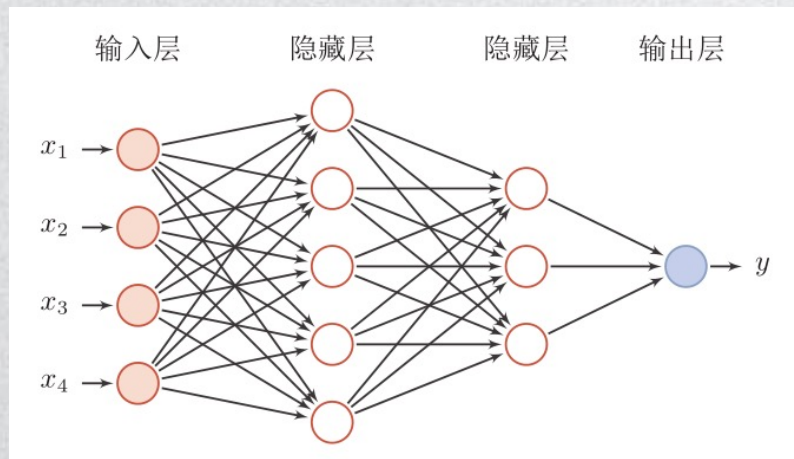
单个神经细胞有两种状态：
抑制或者兴奋



线性模型配上激活函数

神经网络概述

- 最常见的网络结构
 - 前馈神经网络
 - 也称为多层感知机 (MLP) 或者全连接网络



$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)},$$

$$\mathbf{a}^{(l)} = f_l(\mathbf{z}^{(l)}).$$

记号	含义
L	神经网络的层数
M_l	第 l 层神经元的个数
$f_l(\cdot)$	第 l 层神经元的激活函数
$\mathbf{W}^{(l)} \in \mathbb{R}^{M_l \times M_{l-1}}$	第 $l-1$ 层到第 l 层的权重矩阵
$\mathbf{b}^{(l)} \in \mathbb{R}^{M_l}$	第 $l-1$ 层到第 l 层的偏置
$\mathbf{z}^{(l)} \in \mathbb{R}^{M_l}$	第 l 层神经元的净输入 (净活性值)
$\mathbf{a}^{(l)} \in \mathbb{R}^{M_l}$	第 l 层神经元的输出 (活性值)



PyTorch多个线性层

- Pytorch堆积两个线性层（激活函数为sigmoid）

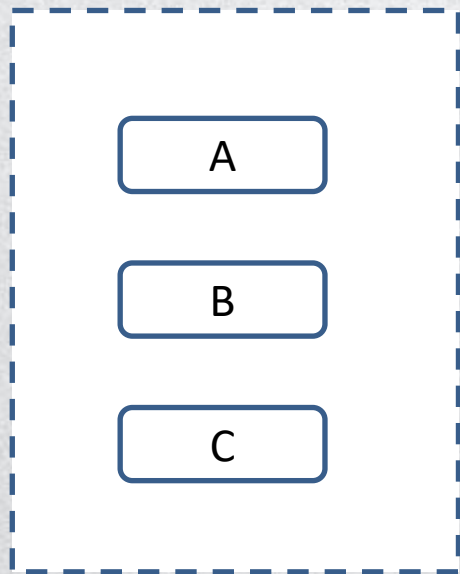
$$z = f(x \cdot W_1 + b_1)$$

$$\hat{y} = f(z \cdot W_2 + b_2)$$

- 如何搭建这样一个网络

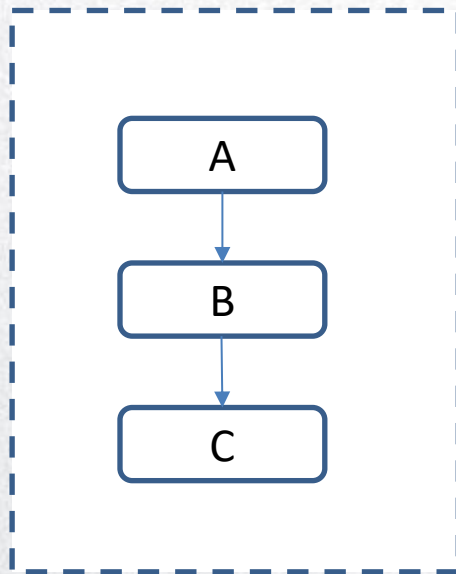
PyTorch多个线性层

- 构造函数与forward方法的“分工”：



`__init__()`

“挂载”成员变量



`forward()`

连同数据流，形成前向计算



● 选零件：PyTorch中提供的“层” (Layer)

- 选择层函数
 - <https://pytorch.org/docs/stable/nn.html>
- 线性层：
 - **nn.Linear**
- 循环层
 - nn.RNN/nn.LSTM/nn.GRU/nn.RNNCell/nn.LSTMCell/nn.GRUCell
- 卷积层
 - nn.Conv1d/ nn.Conv2d/ nn.Conv3d
- 池化层
 - nn.MaxPool1d/ nn.MaxPool2d/nn.MaxPool3d



PyTorch线性层

- `torch.nn.Linear(in_features, out_features, bias=True)`
 - **in_features**: 输入特征的维数，即输入的张量shape: `[data_size, in_dim]` 中的`in_dim`
 - **out_features**: 输出数据的维数，即输出的二维张量的形状为`[data_size, out_dim]`，也代表了该全连接层的神经元个数
 - `data_size`: 输入特征的总数（样本数）
 - **Bias**: 线性变换 $w^T x + b$ 是否加`b`

PyTorch线性层

- 线性层（全连接层）
- $\hat{y} = x \cdot W + b$

$x_1^{(1)}$	$x_1^{(2)}$	...	$x_1^{(d)}$
$x_2^{(1)}$	$x_2^{(2)}$	...	$x_2^{(d)}$
$\vdots$	$\vdots$		$\vdots$
$x_N^{(1)}$	$x_N^{(2)}$	...	$x_N^{(d)}$

$w_1^{(1)}$	$w_1^{(2)}$	...	$w_1^{(d')}$
$w_2^{(1)}$	$w_2^{(2)}$	...	$w_2^{(d')}$
$\vdots$	$\vdots$		$\vdots$
$w_d^{(1)}$	$w_d^{(2)}$	...	$w_d^{(d')}$

+

b_1	b_2	$\vdots$	$b_{d'}$
-------	-------	----------	----------

=

$y_1^{(1)}$	$y_1^{(2)}$	...	$y_1^{(d')}$
$y_2^{(1)}$	$y_2^{(2)}$	...	$y_2^{(d')}$
$\vdots$	$\vdots$		$\vdots$
$y_N^{(1)}$	$y_N^{(2)}$	...	$y_N^{(d')}$

选零件： PyTorch中提供的激活函数

- 选择激活函数

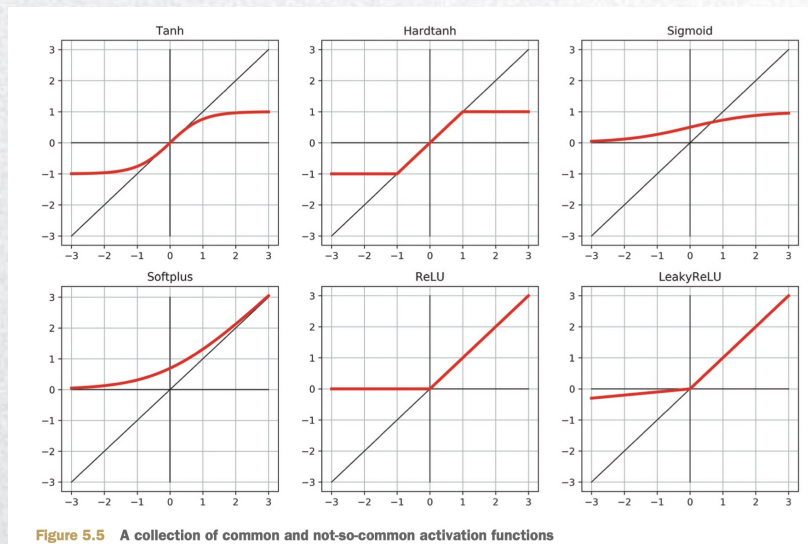
- <https://pytorch.org/docs/stable/nn.html>

- **nn.Sigmoid**

- nn.Tanh

- nn.ReLU

- ...





搭积木：PyTorch实现多个线性层

- PyTorch堆积两个线性层（激活函数为sigmoid）

$$z = f(x \cdot W_1 + b_1)$$

$$\hat{y} = f(z \cdot W_2 + b_2)$$

```
class MLPRegression(nn.Module):
```

```
    def __init__(self, in_dim, hidden_dim):
```

```
        super().__init__()
```

```
        self.first_layer = torch.nn.Linear(in_dim, hidden_dim, bias=True) #set the first layer
```

```
        self.second_layer = torch.nn.Linear(hidden_dim, 1, bias=True) #set the second layer
```

```
        self.sigmoid = torch.nn.Sigmoid()
```

准备“积木”

```
    def forward(self, x):
```

```
        hidden_layer = self.sigmoid(self.first_layer(x))
```

```
        output = self.sigmoid(self.second_layer(hidden_layer))
```

```
        return output
```



搭积木：PyTorch实现多个线性层

- PyTorch堆积两个线性层（激活函数为sigmoid）

$$z = f(x \cdot W_1 + b_1)$$

$$\hat{y} = f(z \cdot W_2 + b_2)$$

```
class MLPRegression(nn.Module):  
    def __init__(self, in_dim, hidden_dim):  
        super().__init__()  
        self.first_layer = torch.nn.Linear(in_dim, hidden_dim, bias=True) #set the first layer  
        self.second_layer = torch.nn.Linear(hidden_dim, 1, bias=True) #set the second layer  
        self.sigmoid = torch.nn.Sigmoid()  
  
    def forward(self, x):  
        hidden_layer = self.sigmoid( self.first_layer(x) )  
        output = self.sigmoid( self.second_layer(hidden_layer) )  
        return output
```

搭“积木”

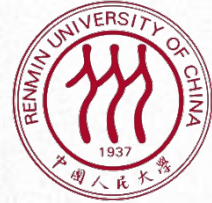


问题：全连接网络的参数的个数？

- 考虑一层全连接网络：

$$z = f(x \cdot W_1 + b_1)$$

- 假设 x 的维度为 m
 - z 的维度（神经元个数）为 n
- 一层全连接网络的参数个数为？



提纲



图像与卷积

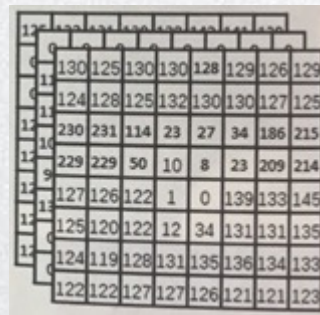
- ☐ 图像数据
- ☐ 图像卷积操作
-
-

数字图像

- 数字图像是连续的光信号经过传感器的采样在空间域上的表达。一张图像是由一个包含若干个像素点的矩形框组成的。
- 每个小格子是一个像素
- 灰度值需要1个维度，表示像素的灰度值
- 彩色值需要3个维度，也就是3个图像通道：每个像素3个数字表示。



我们看到的图像



计算机看到的图像



数字图像

- 图像分辨率 (Resolution) : 水平方向的像素数 * 画面垂直方向的像素数。例如500x200, 水平方向有500个像素点, 垂直方向有200个像素点; 数码像素点是虚拟化的, 没有实际的物理尺寸大小
- Python读取图像后, 得到矩阵的大小: $H*W*3$
- 数码打印/屏幕/打印机分辨率: 像素每英寸, 单位为PPI(Pixels Per Inch) 每英寸多少个屏幕像素点; 屏幕像素具有尺寸大小 (如LED屏幕)
- 如果用像素数表示屏幕, 如 1024×768 , 表示设备屏幕的水平方向上有1024个屏幕像素点, 垂直方向上有768个屏幕像素点

图像

- Python自带图像处理库PIL读取图片
- 读取转numpy, numpy转tensor



```
<class 'PIL.PngImagePlugin.PngImageFile'>  
(512, 512, 3)  
torch.Size([3, 512, 512])  
(512, 512, 3)
```

```
import torch  
import torch.nn as nn  
import numpy as np  
from PIL import Image  
from torchvision import transforms  
import torch.nn.functional as F  
import matplotlib.pyplot as plt
```

```
image_name = 'lenaOriginal.png'  
img = Image.open(image_name)  
print(type(img))
```

```
img = np.array(img)  
print(img.shape) #转numpy
```

```
img = torch.from_numpy(img.transpose((2, 0, 1)))  
print(img.shape)  
img = img.numpy().transpose((1, 2, 0))  
print(img.shape)
```




Pytorch处理图像数据

- torchvision是pytorch的计算机视觉工具包，包括三个主要的模块：
- torchvision.transforms，常用的图像预处理方法，在transforms中提供了一系列的图像预处理方法，例如数据的标准化，中心化，旋转，翻转等
- torchvision.datasets，定义了一系列常用的公开数据集的datasets，比如常用的MNIST，CIFAR-10，ImageNet等
- torchvision.model，提供大量常用的预训练模型，例如AlexNet，VGG，ResNet，GoogLeNet等



Pytorch处理图像数据

- 将PIL读取的图像转成tensor: ToTensor 类, 将 PIL.Image 转换为 torch.Tensor 类型并归一化到 $[0.0, 1.0]$ 之间
 - `transforms.ToTensor()`
 - 取值范围为 $[0, 255]$ 的PIL.Image, 转换成形状为 $[C, H, W]$, 取值范围是 $[0, 1.0]$ 的`torch.FloatTensor`
- Tensor转PIL
 - `transforms.ToPILImage()`



Pytorch处理图像数据

```
# 使用torchvision中自带的transforms
image = Image.open(image_name).convert('RGB')
image = transforms.ToTensor()(image).unsqueeze(0)
print(image.shape)

image = image.squeeze(0)
image = transforms.ToPILImage()(image)
print(type(image))

totensor = transforms.ToTensor()
toPIL = transforms.ToPILImage()

torch.Size([1, 3, 512, 512])
<class 'PIL.Image.Image'>
```




图像预处理

- torchvision.transform 模块中的各种类，要求传入的图片为 PIL.Image 类型的图片
- torchvision.transforms: 常用的图像预处理方法
 - 数据中心化、数据标准化
 - 缩放、裁剪
 - 旋转、翻转、填充
 - 噪声添加
 - 灰度变换
 - 线性变换、仿射变换
 - 亮度、饱和度及对比度变换



标准化



- `transforms.Normalize`
- 功能：逐channel的对图像进行标准化，即数据的均值变为0，标准差变为1
- 标准化的计算公式为 $\text{output} = (\text{input} - \text{mean}) / \text{std}$
- `mean`：各通道的均值
- `std`：各通道的标准差
- `inplace`：是否原位操作

标准化

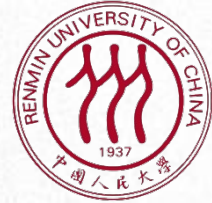
```
# 图像预处理 normalization
norm_mean = [0.485, 0.456, 0.406]
norm_std = [0.229, 0.224, 0.225]

normalize = transforms.Normalize(
    mean=norm_mean,
    std=norm_std,
    inplace=False
)

image_nor = normalize(totensor(image))
print(image_nor.mean(), image_nor.std())
image_nor = toPIL(image_nor)
#image.save("lena_normalized.jpg")
image_nor.show()

tensor(-0.4199) tensor(1.0752)
```





提纲



图像与卷积

- ☐ 图像数据
- ☐ 图像卷积操作
-
-



卷积

- 图像数据量
 - 64×64 的小图片，数据量是 $64 \times 64 \times 3$
 - 1000×1000 的图片（1M），数据量达到了 $1000 \times 1000 \times 3$ （3M）
 - 输入300万的数据量，特征向量的维度高达300万
 - 1000个隐节点的线性层（全连接网络），参数高达 $1000 \times 3M$ ，即30亿个参数
 - 内存开销，过拟合
- 处理大图需要卷积运算

卷积运算

- 例：边缘检测
 - 给定一张图，让电脑去搞清楚这张照片里有什么物体，可能首先需要检测图片中的边缘



卷积运算

- 例：检测如下 6×6 的灰度图像的垂直边缘

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

- 构造一个 3×3 矩阵
- 滤波器
- 卷积核

$$\begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

卷积运算

- 输出：4*4
- 第一步

3 ¹	0 ⁰	1 ⁻¹	2	7	4
1 ¹	5 ⁰	8 ⁻¹	9	3	1
2 ¹	7 ⁰	2 ⁻¹	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

*

1	0	-1
1	0	-1
1	0	-1

=

卷积运算

- 右移一步

3	0 ¹	1 ⁰	2 ⁻¹	7	4
1	5 ¹	8 ⁰	9 ⁻¹	3	1
2	7 ¹	2 ⁰	5 ⁻¹	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

*

1	0	-1
1	0	-1
1	0	-1

=

-5			

卷积运算

- 继续移动

3	<u>0</u>	<u>1</u> ¹	<u>2</u> ⁰	<u>7</u> ⁻¹	4
1	<u>5</u>	<u>8</u> ¹	<u>9</u> ⁰	<u>3</u> ⁻¹	1
2	<u>7</u>	<u>2</u> ¹	<u>5</u> ⁰	<u>1</u> ⁻¹	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

*

1	0	-1
1	0	-1
1	0	-1

=

<u>-5</u>	<u>-4</u>		

卷积运算

3	<u>0</u>	<u>1</u>	<u>2</u> ¹	<u>7</u> ⁰	<u>4</u> ⁻¹
1	<u>5</u>	<u>8</u>	<u>9</u> ¹	<u>3</u> ⁰	<u>1</u> ⁻¹
2	<u>7</u>	<u>2</u>	<u>5</u> ¹	<u>1</u> ⁰	<u>3</u> ⁻¹
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

*

1	0	-1
1	0	-1
1	0	-1

=

<u>-5</u>	<u>-4</u>	0	8

卷积运算

- 把蓝色块下移，从左开始

3	<u>0</u>	<u>1</u>	<u>2</u>	<u>7</u>	<u>4</u>
¹ 1	⁰ <u>5</u>	⁻¹ <u>8</u>	<u>9</u>	<u>3</u>	<u>1</u>
¹ 2	⁰ <u>7</u>	⁻¹ <u>2</u>	<u>5</u>	<u>1</u>	<u>3</u>
¹ 0	⁰ 1	⁻¹ 3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

*

1	0	-1
1	0	-1
1	0	-1

=

<u>-5</u>	<u>-4</u>	0	<u>8</u>
-			

卷积运算

- 直到最后

3	<u>0</u>	<u>1</u>	<u>2</u>	<u>7</u>	<u>4</u>
1	<u>5</u>	<u>8</u>	<u>9</u>	<u>3</u>	<u>1</u>
2	<u>7</u>	<u>2</u>	<u>5</u>	<u>1</u>	<u>3</u>
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

*

1	0	-1
1	0	-1
1	0	-1

=

-5	-4	0	8
-10	-2	2	3
0	-2	-4	-7
-3	-2	-3	-16

练习：垂直边缘检测

- 计算卷积结果

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0

*

1	0	-1
1	0	-1
1	0	-1

A diagram showing a vertical edge. It consists of a white rectangle on the left and a gray rectangle on the right, separated by a vertical blue line. This represents the result of a vertical edge detection filter applied to the input image.

垂直边缘检测

- 图像中间有一个特别明显的垂直边缘

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0



*

1	0	-1
1	0	-1
1	0	-1



=

0	30	30	0
0	30	30	0
0	30	30	0
0	30	30	0



垂直边缘检测

- 颜色被翻转了，变成了左边比较暗，而右边比较亮
- 区分这两种明暗变化的区别

0	0	0	10	10	10
0	0	0	10	10	10
0	0	0	10	10	10
0	0	0	10	10	10
0	0	0	10	10	10
0	0	0	10	10	10



*

1	0	-1
1	0	-1
1	0	-1



=

0	-30	-30	0
0	-30	-30	0
0	-30	-30	0
0	-30	-30	0



水平边缘检测

- 水平边缘检测：滤波器

1	1	1
0	0	0
-1	-1	-1

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
0	0	0	10	10	10
0	0	0	10	10	10
0	0	0	10	10	10

*

1	1	1
0	0	0
-1	-1	-1

=

0	0	0	0
30	10	-10	-30
30	10	-10	-30
0	0	0	0

水平边缘检测

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
0	0	0	10	10	10
0	0	0	10	10	10
0	0	0	10	10	10

*

1	1	1
0	0	0
-1	-1	-1

=

0	0	0	0
30	10	-10	-30
30	10	-10	-30
0	0	0	0

边缘检测

- 通过使用不同的过滤器，可以找出垂直的或是水平的边缘
- 不同的数字组合可以实现不同的目标
- 实现相同的目标也用不同的数字组合
- 在计算机视觉的文献中，曾争论过怎样的数字组合才是最好的

- Sobel滤波器：增加中间一行的权重，更鲁棒

$$\begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}$$

- Scharr滤波器

$$\begin{bmatrix} 3 & 0 & -3 \\ 10 & 0 & -10 \\ 3 & 0 & -3 \end{bmatrix}$$

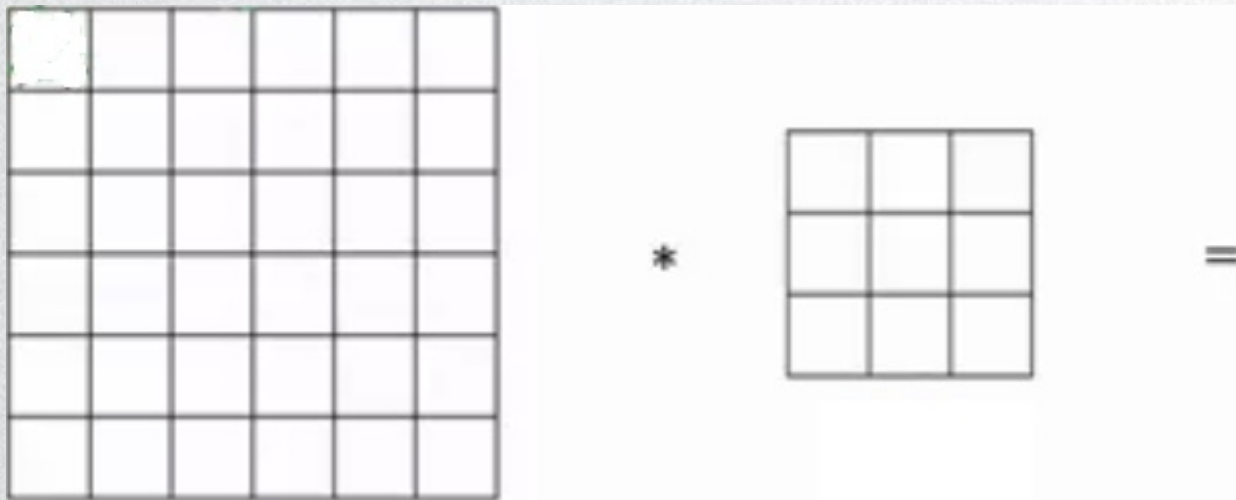
滤波器

- 边缘检测不一定要去使用研究者们所选择的这九个数字
- 检测复杂图像的边缘
- 数据和任务不同，需要的滤波器可能也不同



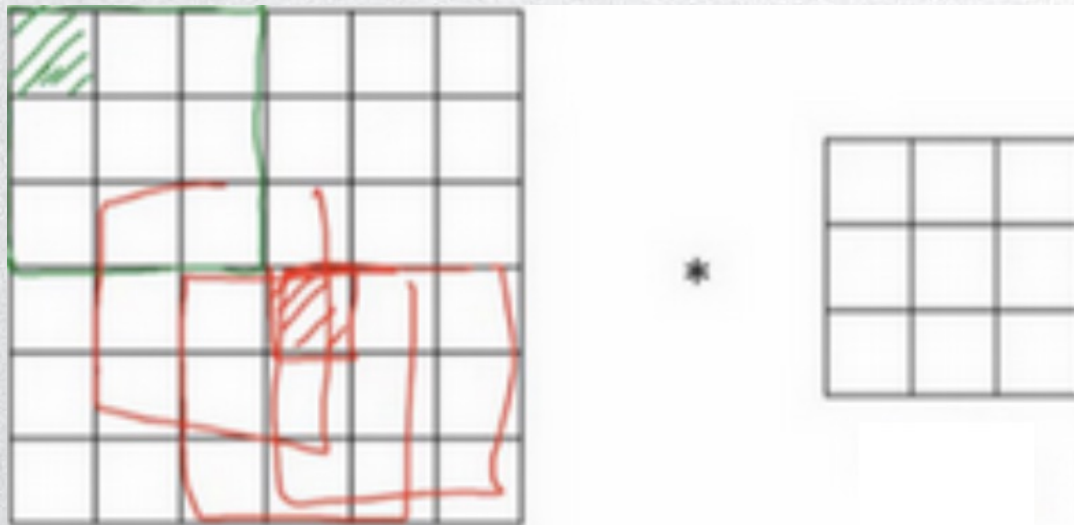
Padding

- 用 3×3 的滤波器对一个 6×6 的图像进行卷积，得到一个 4×4 的输出
- $n \times n$ 的图像，用 $f \times f$ 的滤波器进行卷积，输出维度： $(n-f+1) \times (n-f+1)$
- 缺点：每次做卷积，图像会缩小



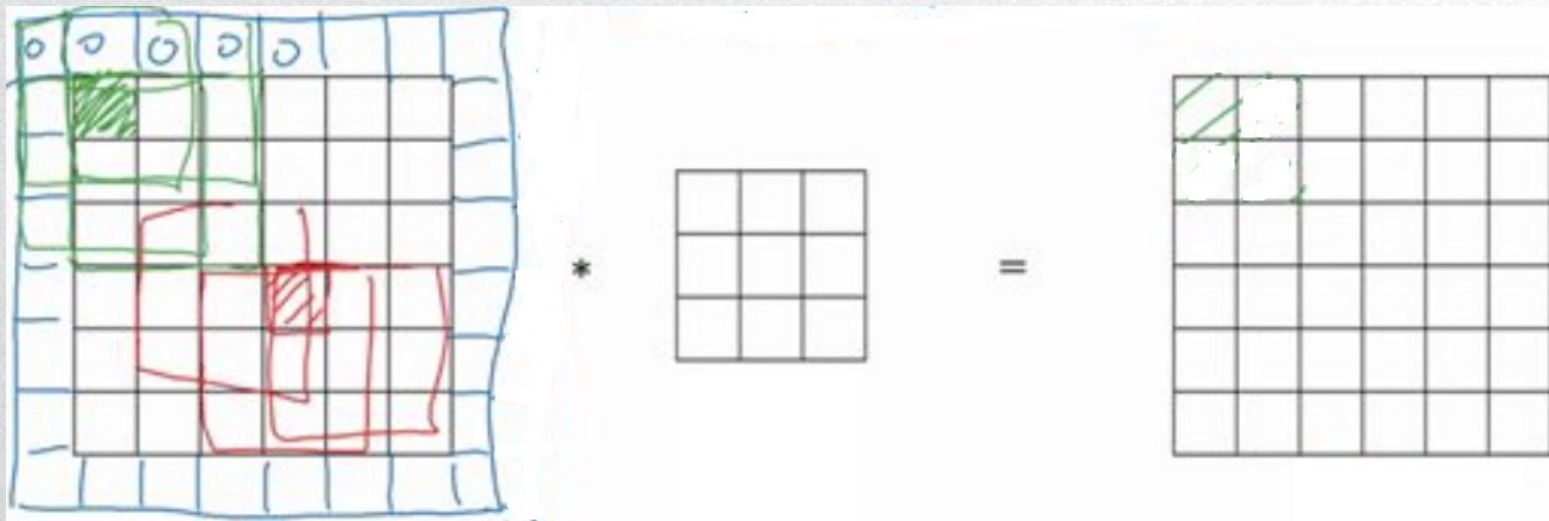
Padding

- 角落边缘的像素只被一个 3×3 的区域所触碰或者使用；中间的像素点会有许多 3×3 的区域与之重叠。角落或者边缘区域的像素点在输出中采用较少，意味着丢掉了图像边缘位置的许多信息



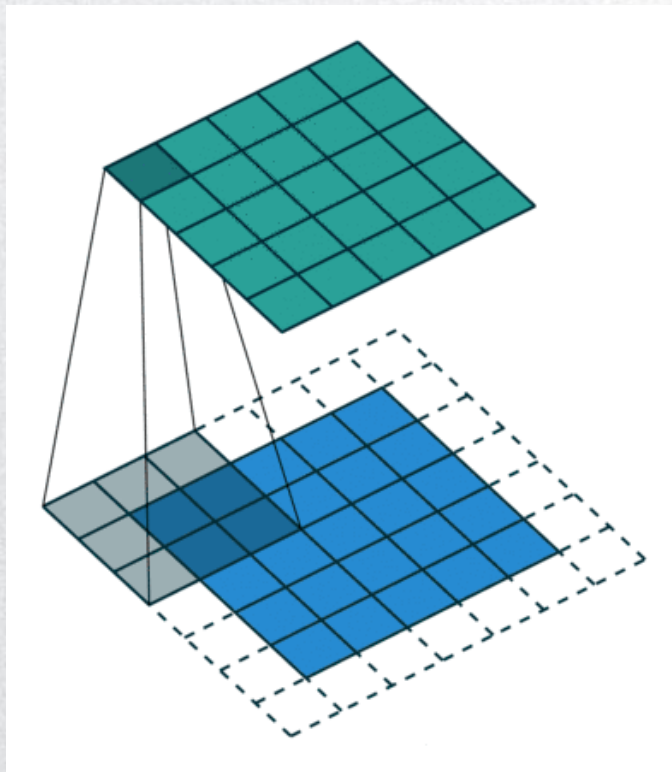
Padding

- 在卷积操作之前填充这幅图像：沿着图像边缘再填充一层像素
- 6*6输入padding为8*8图像，卷积后得到6*6输出
- 通常可以用0去填充



Padding

- Padding后卷积





Padding

- 可以任意填充p层, 输出: $(n+2p-f+1)*(n+2p-f+1)$
- 常用两种Padding方式: Valid卷积和Same卷积
- Valid卷积: 不填充, $p=0$
- Same卷积: 填充后, 卷积输出大小和原始输入大小一样
 - $n+2p-f+1=n$
 - $p=(f-1)/2$
 - 因此卷积核大小通常是奇数

步长stride

- 卷积中的步长 (stride) 是另一个构建卷积神经网络的基本操作

2 ³	3 ⁴	7 ⁴	4	6	2	9
6 ¹	6 ⁰	9 ²	8	7	4	3
3 ⁻¹	4 ⁰	8 ³	3	8	9	7
7	8	3	6	6	3	4
4	2	1	8	3	4	6
3	2	4	1	9	8	3
0	1	3	9	2	1	4

*

3	4	4
1	0	2
-1	0	3

=

91.		

步长stride

- 步长stride为2

2	3	7	4	6	2	9
6	6	9	8	7	4	3
3	4	8	3	8	9	7
7	8	3	6	6	3	4
4	2	1	8	3	4	6
3	2	4	1	9	8	3
0	1	3	9	2	1	4

*

3	4	4
1	0	2
-1	0	3

=

91	100	

步长stride

- 步长stride为2

2	3	7	4	6 ³	2 ⁴	9 ⁴
6	6	9	8	7 ¹	4 ⁰	3 ²
3	4	8	3	8 ⁻¹	9 ⁰	7 ³
7	8	3	6	6	3	4
4	2	1	8	3	4	6
3	2	4	1	9	8	3
0	1	3	9	2	1	4

*

3	4	4
1	0	2
-1	0	3

=

91	100	83

步长stride

- 步长stride为2

2	3	7	4	6	2	9
6	6	9	8	7	4	3
3 ³	4 ⁴	8 ⁴	3	8	9	7
7 ¹	8 ⁰	3 ²	6	6	3	4
4 ⁻¹	2 ⁰	1 ³	8	3	4	6
3	2	4	1	9	8	3
0	1	3	9	2	1	4

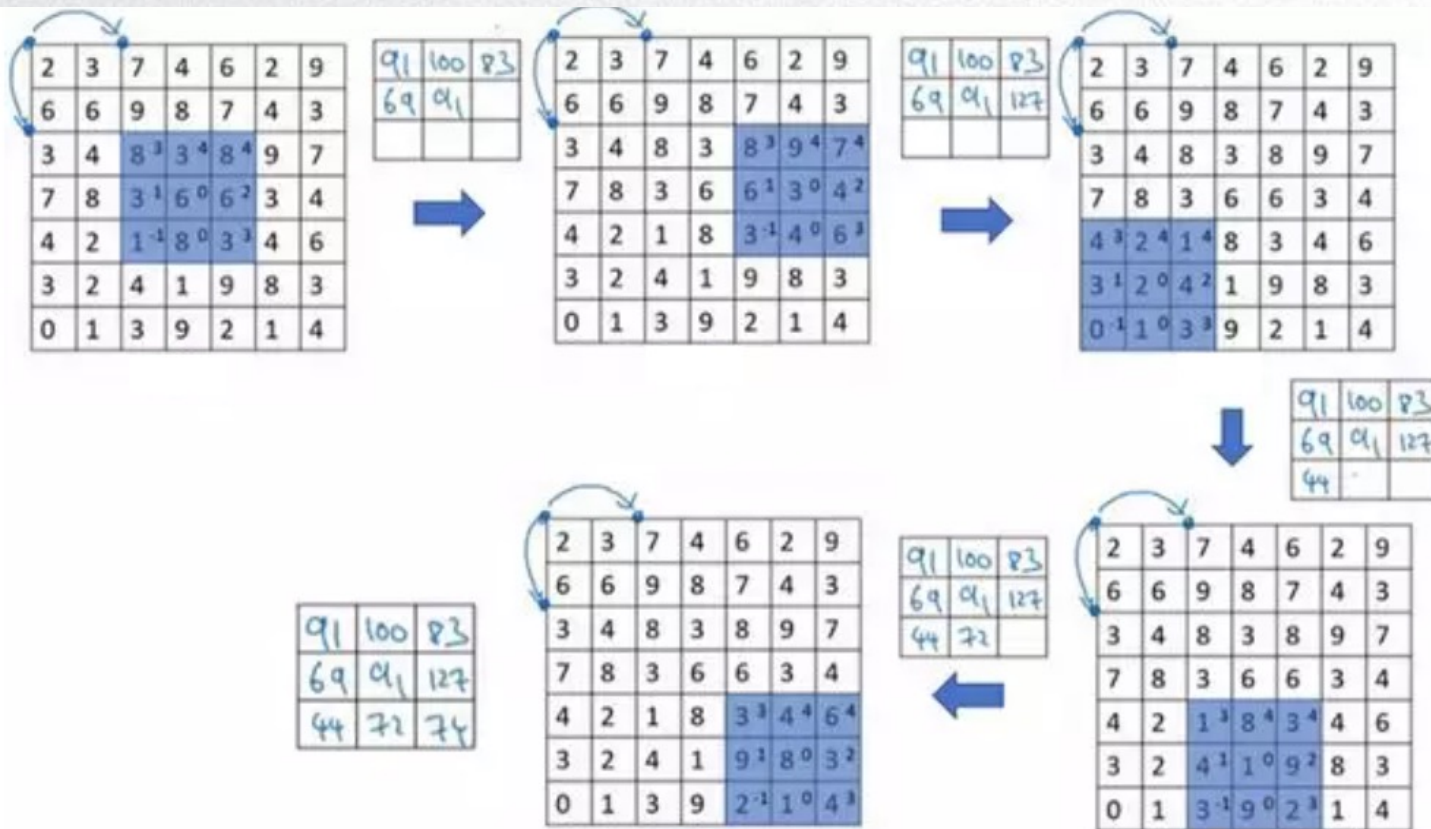
*

3	4	4
1	0	2
-1	0	3

=

91	100	83
69		

步长stride



步长stride

- $n \times n$ 的图像，用 $f \times f$ 的滤波器进行卷积，padding= p ，步长设为 s ，
- 输出大小

$$\frac{n+2p-f}{s} + 1 \times \frac{n+2p-f}{s} + 1$$

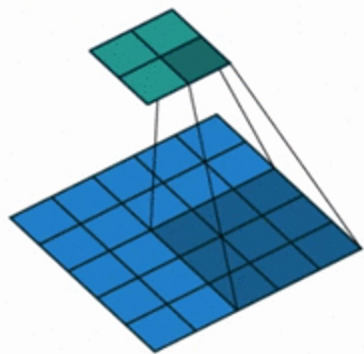
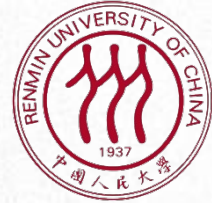
- 如果商不是一个整数，向下取整
- 原则：滤波必须完全处于图像中或者填充之后的图像区域内才输出相应结果

$n \times n$ image $f \times f$ filter

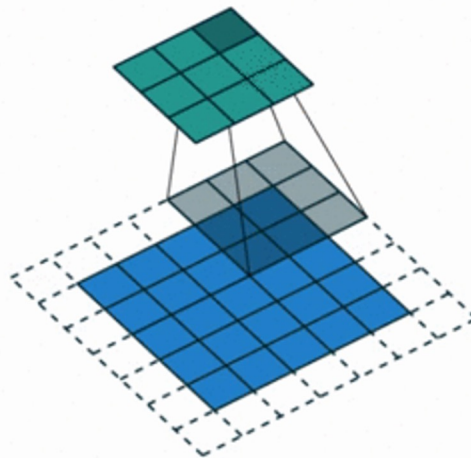
padding p stride s

$$\left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor$$

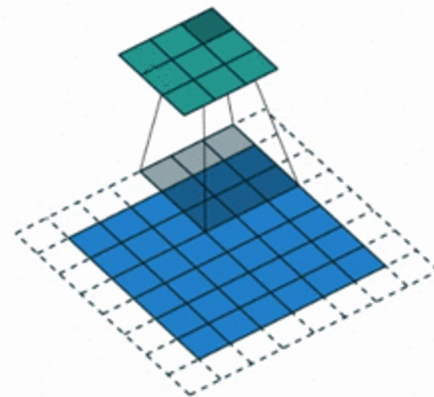
卷积示例



No padding, strides



Padding, strides



Padding, strides (odd)



互相关与卷积

- 数学或信号处理教科书：卷积在做元素乘积求和之前还有一个步骤，即先将滤波器沿水平和垂直轴翻转
- 用翻转后的滤波器进行滑动并在每个位置对元素乘机求和
- 卷积定义包含翻转操作，则卷积满足结合律 $(A*B)*C=A*(B*C)$
- 深度学习中这个性质不重要，因此省略掉这个双镜像操作
- 从技术上说，省略后的卷积可能叫做互相关更好



Pytorch实现卷积

- `torch.nn.functional.conv2d(input, weight, bias=None, stride=1, padding=0, dilation=1, groups=1)`
 - `input`: 输入张量 (batchsize, in_channels, iH, iW) , iH和iW为输入tensor的高、宽
 - `weight`: 过滤器 (卷积核) 张量 (out_channels, in_channels/groups, kH, kW) , kH和kW为卷积核的高、宽
 - `bias`: 可选偏置张量 (out_channels)
 - `stride`: 卷积核的步长, 可以是单个数字或一个元组 (sh x sw)
 - `padding`: 输入上隐含零填充。可以是单个数字或元组
 - `groups`: 将输入分成组, in_channels应该被组数除尽



Pytorch实现卷积

- `torch.nn.functional.conv2d(input, weight, bias=None, stride=1, padding=0, dilation=1, groups=1)`

卷积

```
x = torch.tensor([[[[3., 0., 1., 2., 7., 4.],
→          [1., 5., 8., 9., 3., 1.],
→          [2., 7., 2., 5., 1., 3.],
→          [0., 1., 3., 1., 7., 8.],
→          [4., 2., 1., 6., 2., 8.],
→          [2., 4., 5., 2., 3., 9.]]]], requires_grad=False)

w = torch.tensor([[[[1., 0., -1.],
→          [1., 0., -1.],
→          [1., 0., -1.]]]], requires_grad=True)
```

```
out = F.conv2d(x, w, stride=1, padding=0)
print(out)
```

```
out1 = F.conv2d(x, w, stride=1, padding=1)
print(out1)
```

```
out2 = F.conv2d(x, w, stride=2, padding=0)
print(out2)
```

```
tensor([[[[ -5.,  -4.,   0.,   8.],
            [-10., -2.,   2.,   3.],
            [  0., -2.,  -4.,  -7.],
            [-3., -2.,  -3., -16.]]]], grad_fn=<ThnnConv2DBackward>)
tensor([[[[ -5.,  -5.,  -6.,  -1.,   6.,  10.],
            [-12., -5.,  -4.,   0.,   8.,  11.],
            [-13., -10., -2.,   2.,   3.,  11.],
            [-10.,   0., -2.,  -4.,  -7.,  10.],
            [-7.,  -3., -2.,  -3., -16.,  12.],
            [-6.,   0., -2.,   1.,  -9.,   5.]]]],
        grad_fn=<ThnnConv2DBackward>)
tensor([[[[-5.,   0.],
            [ 0., -4.]]]], grad fn=<ThnnConv2DBackward>)
```

Pytorch实现卷积

灰度图卷积: 边缘检测

```
grayimage = image.convert('L')  
grayimage = totensor(grayimage).unsqueeze(0)  
w = torch.tensor([[[[1., 0., -1.],  
——x——x——x——x [2., 0., -2.],  
——x——x——x——x [1., 0., -1.]]]])  
convv = F.conv2d(grayimage, w, stride=1, padding=0)  
convv = convv.squeeze(0)  
convv = toPIL(convv)  
convv.show()
```





Pytorch实现卷积



```
# padding
```

```
convpad = F.conv2d(grayimage, w, stride=1, padding=10)
```

```
convpad = convpad.squeeze(0)
```

```
convpad = toPIL(convpad)
```

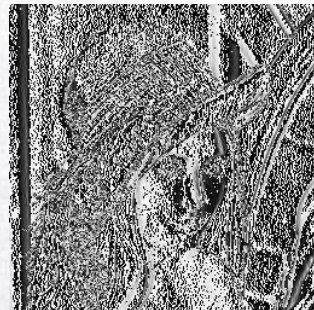
```
convpad.show()
```



Pytorch实现卷积

步长

```
convstride = F.conv2d(grayimage, w, stride=2, padding=0)
convstride = convstride.squeeze(0)
convstride = toPIL(convstride)
convstride.show()
```



Pytorch实现卷积

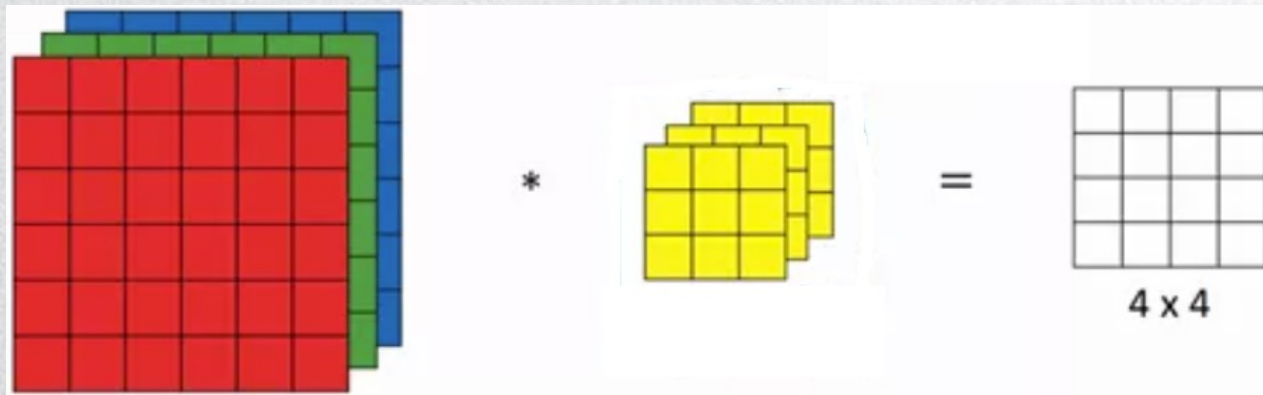
水平线检测

```
w2 = torch.tensor([[[[1., 2., 1.],  
——x——x——x——x [0., 0., 0.],  
——x——x——x——x [-1., -2., -1.]]]]])  
convh = F.conv2d(grayimage, w2, stride=1, padding=0)  
convh = convh.squeeze(0)  
convh = toPIL(convh)  
convh.show()
```



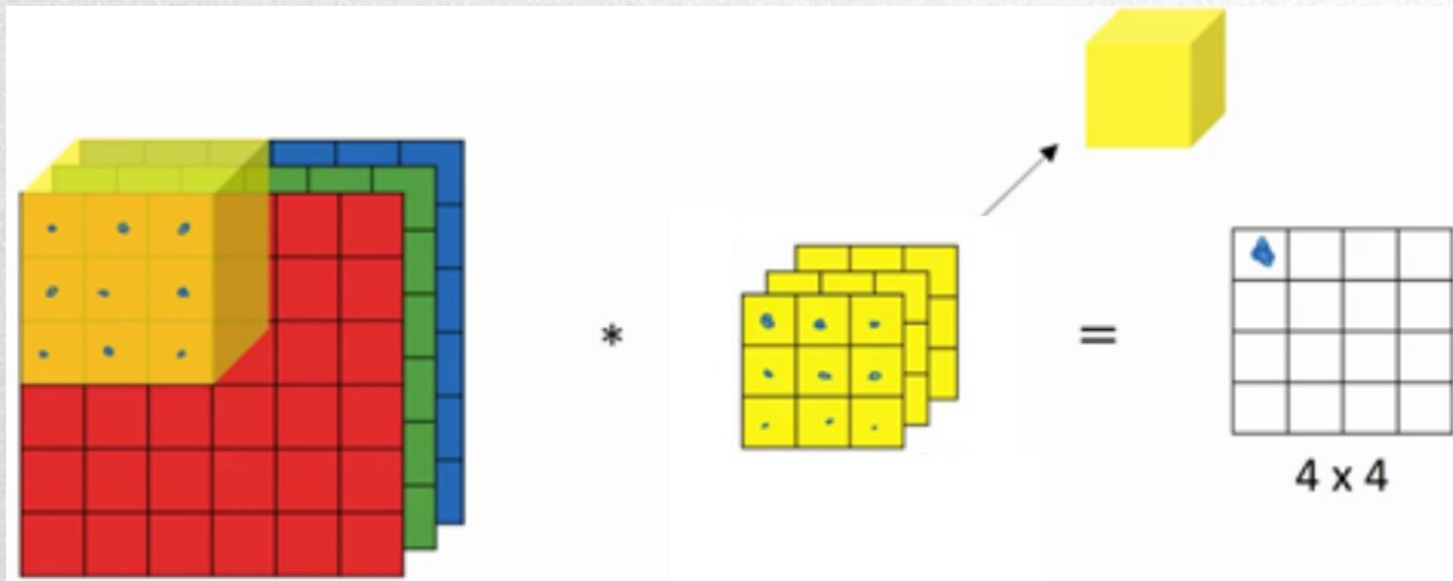
多通道卷积

- 检测RGB彩色图像的特征
- 例：彩色图像如果是 $6 \times 6 \times 3$ （高度 $\times$ 宽度 $\times$ 通道数），这里的3指的是三个颜色通道，可以把它想象成三个 6×6 灰度图像的堆叠。
- 为了检测图像的边缘或者其他特征，不是把它跟原来的 3×3 的过滤器做卷积，而是跟一个三维的过滤器，它的维度是 $3 \times 3 \times 3$ ，这样这个过滤器也有三层，对应红、绿、蓝三个通道。

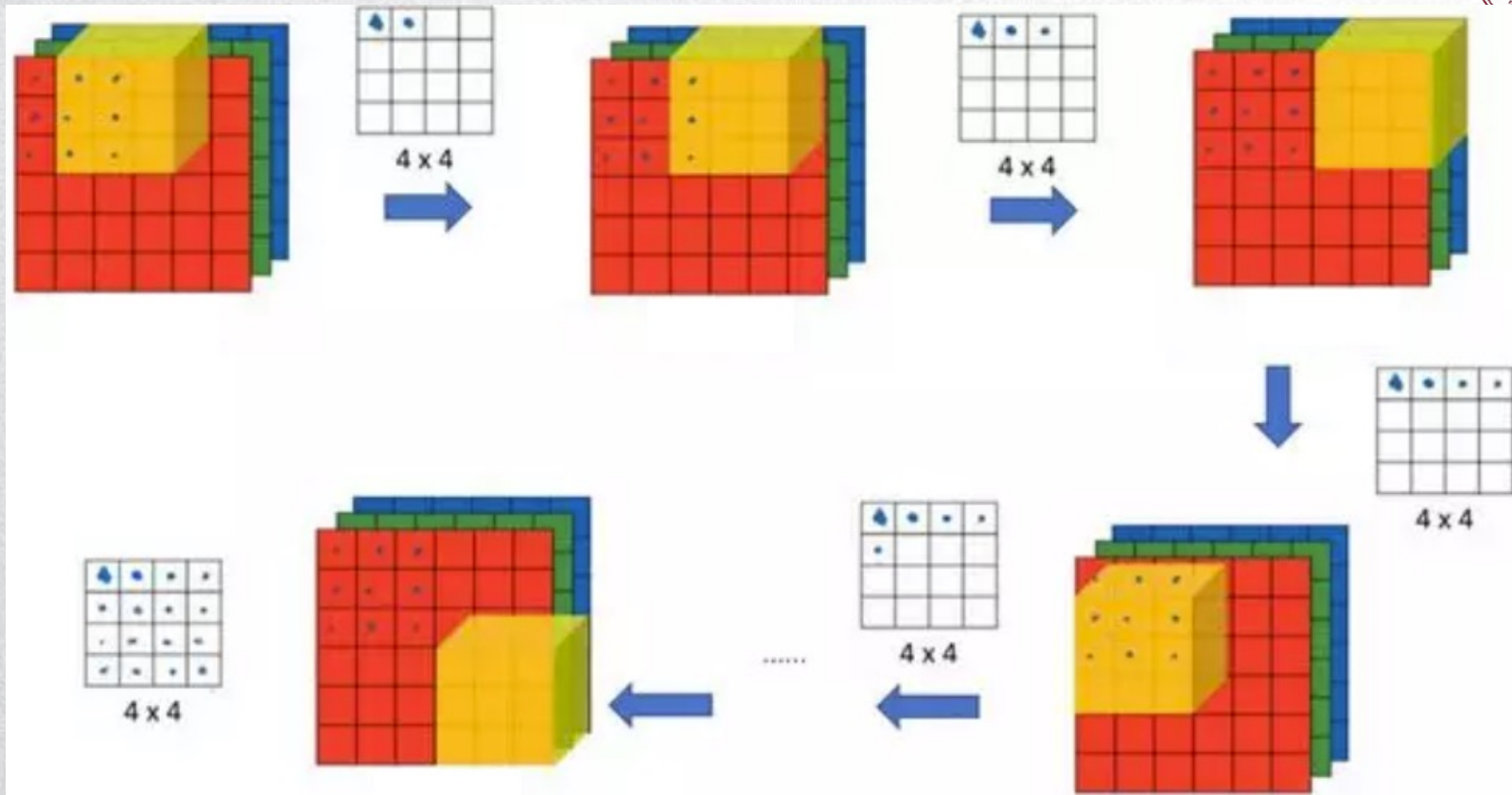


多通道卷积

- 滤波器通道数和图像通道数要匹配
- 取出27个像素值，依次和滤波器对应参数相乘，求和



多通道卷积





多通道卷积

- 参数的选择不同，得到不同的特征检测器
- 当输入有特定的高宽和通道数时，滤波器可以有不同的高，不同的宽，但是必须一样的通道数
- padding、stride、输出维度
- 多通道卷积：输出的通道数会等于要检测的特征数

多通道卷积

多通道卷积 (RGB)

```
image = totensor(image).unsqueeze(0)
```

```
w = torch.tensor([[[[1., 0., -1.],  
——x——x——x——x [2., 0., -2.],  
——x——x——x——x [1., 0., -1.]]]])
```

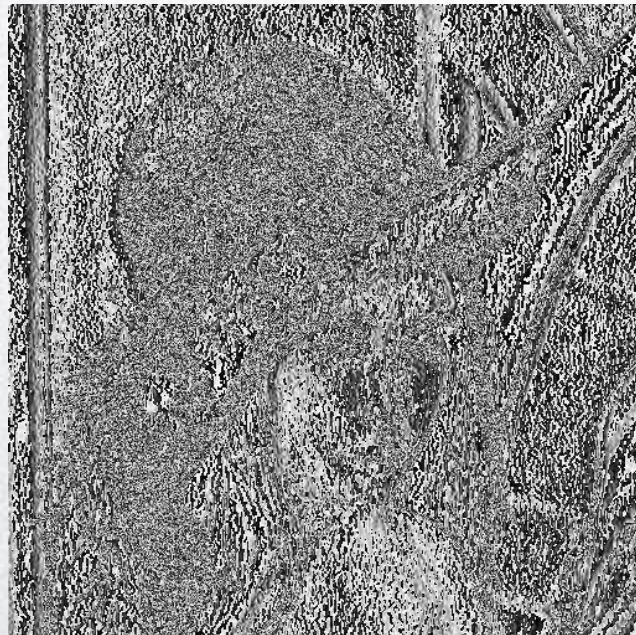
```
w = w.repeat(1, 3, 1, 1)
```

```
convv = F.conv2d(image, w, stride=1, padding=0)
```

```
convv = convv.squeeze(0)
```

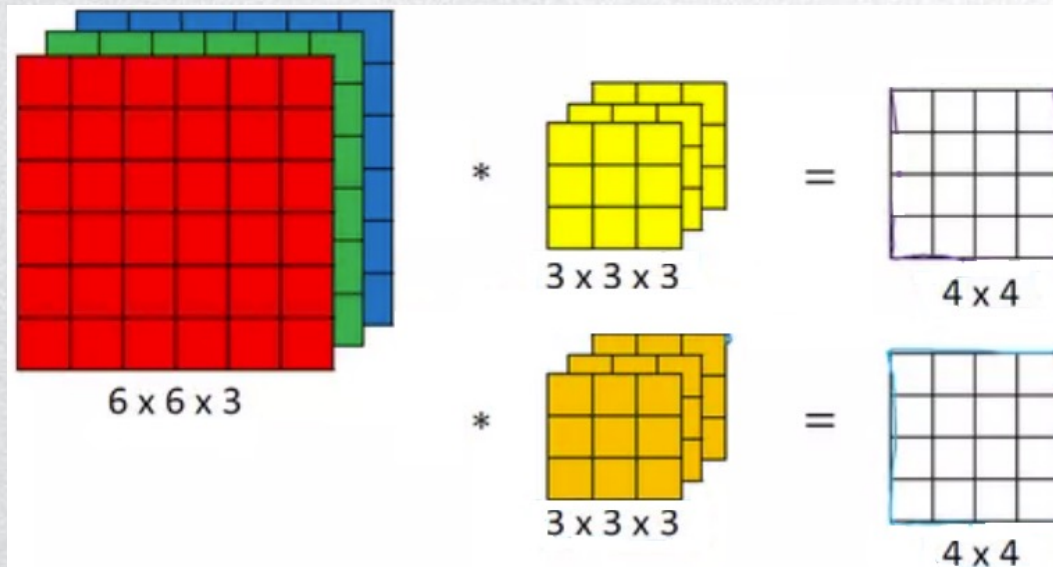
```
convv = toPIL(convv)
```

```
convv.show()
```



多卷积核

- 不仅检测垂直边缘，还需要同时检测垂直边缘和水平边缘
- 同时用多个过滤器：把输出堆叠在一起，得到 $4 \times 4 \times 2$ 的输出立方体



多卷积核

多卷积核

```
w2 = torch.tensor([[[[1., 2., 1.],  
——x——x——x——x [0., 0., 0.],  
——x——x——x——x [-1., -2., -1.]]]]])
```

```
w2 = w2.repeat(1, 3, 1, 1)
```

```
w3 = torch.cat((w, w2), 0)
```

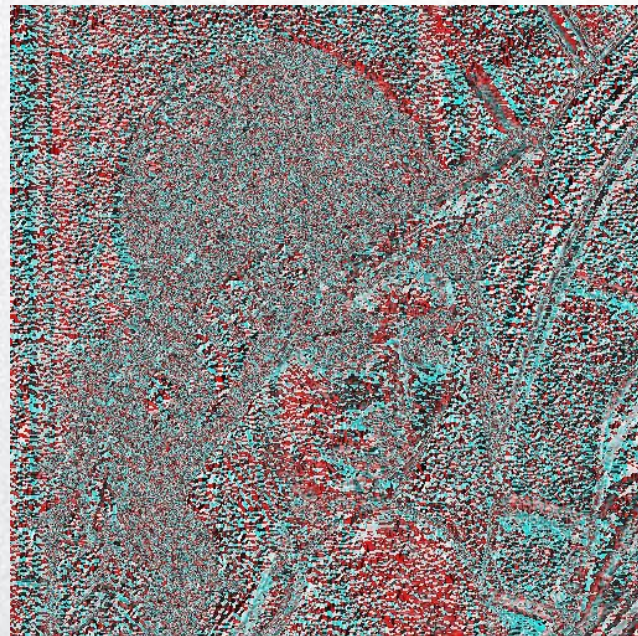
```
w3 = torch.cat((w3, w2), 0)
```

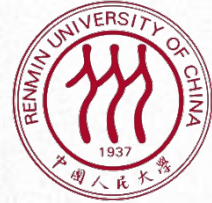
```
convh = F.conv2d(image, w3, stride=1, padding=0)
```

```
convh = convh.squeeze(0)
```

```
convh = toPIL(convh)
```

```
convh.show()
```





谢谢！